

Knowhow：人形机器人 Diffusion Policy 包裹处理

作者： 陈彦宇 · 周期： 2026 年 5 月 25 日 – 7 月 31 日 · 团队： MUJIN 交接状态： 端到端双臂演示可正常运行，尚未达到量产水平
代码： <https://gitlab.mujin.com.cn/app/humanoid/-/tree/new-inference> · 配套文档： [Humanoid_功能演示](#)、[Humanoid_技术验证报告](#)

0. 如何阅读本文档

二十分钟速览： §3（宏观模型）→ §7.2（振荡问题始末）。

第一天： 在上述基础上补充 §2（把系统跑起来）、§4（模块说明）、§8（可调参数）。

改动运动流程之前： 务必先通读 §7 全节。

1. 系统功能概要

机器人用双臂从台面抓起包裹并抬升，判断快递单是否可见；看不到就把包裹翻转约 180°，最后以快递单朝上的姿态放好。

本代码是围绕 **diffusion policy**（Chi et al., Stanford REALab）训练框架搭建的机器人控制系统：训练数据由 VR 遥操作教采集，policy 以端到端方式完成从视觉感知到抓取、抬升、翻转、放置、释放的完整任务链路；失败判定、失败恢复与启停判定则由传统逻辑配合 YOLO 感知实现。

本代码**不包含 diffusion policy 本体**。policy 只负责读取我们给出的观测字典、输出 action chunk，与机器人本体没有直接耦合；本代码承担的是它周围的全部物理部署工作——遥操作、录制、dataset 构建、执行下发，以及配套的一整套安全机制（watchdog 等）。

2. 平台与启动方式

项目	值	位置
机器人	AgiBot / 智元 精灵 G1 双臂人形机器人	–
厂商 SDK	a2d_sdk ，随仓库内置；GDK v1.5.0 手册位于 docs/	a2d_sdk/robot.py
相机	头部 + 双腕（ head 、 hand_left 、 hand_right ）	real_world/camera.py:31
Policy	Diffusion policy；图像条件化、双臂、末端执行器空间	服务器端
运动学	Pinocchio + 自有 URDF， 非 SDK 自带求解器	real_world/ik.py
仿真	PyBullet（进程内）	real_world/sim_backend.py
录制频率	RECORD_HZ = 10 Hz：policy 的 action row 节拍	real_world/timing.py:44
控制频率	CONTROL_HZ = 120 Hz：向机械臂下发子步的频率	real_world/timing.py:46
遥操作	Pico VR 头显，经由 ZeroMQ 通信	pico_vr/
Policy 服务器	10.12.11.144:9000 （脚本默认 :9001 ）	real_world/inference_controller.py:60

命名需注意。 厂商包名为 [a2d_sdk](#)，厂商手册封面标题为「G01 GDK」，机器人型号是 AgiBot Genie G1，URDF 文件名为 [A2D.urdf](#)。磁盘上还遗留一份历史快照 [G1_SDK_ENV/](#)，已排除在版本控制之外，无需关注。

如何运行

```
# 1) Robot machine – full stack (needs a2d_sdk + ROS + the robot):
pip install -r requirements_gui.txt          # + `pip install ultralytics` for the YOLO gates
python robot_control_gui.py
```

```
# 2) Any laptop - hardware-free demo mode (synthetic robot, live webcam, no SDK/ROS):
python robot_control_gui.py --demo

# 3) SDK-free sim / IK tools:
python -m venv .venv && .venv/bin/pip install -r requirements_sim.txt
.venv/bin/python scripts/sim_infer_eval.py --source replay --recording recording001 \
    --recordings MDM_data_collection/recordings

# 4) Tests (also run automatically as a launch pre-flight - see below):
pytest tests/
```

所有命令一律在仓库根目录下执行，否则 `real_world`、`gui`、`servers`、`a2d_sdk` 无法正确解析。

启动前自检。 `robot_control_gui.py:391` 会在 GUI 构建之前先运行一遍 `tests/test_safety_invariants.py`，一旦出现回归即阻止启动。

GUI (`robot_control_gui.py`，四个页签)

- **Console**：相机画面与推理面板。标准操作顺序已直接标注在界面上 (`gui/inference_panel.py:667`)：① *Start sim preview* → ② *Start auto-run*。注意：出于安全考虑，未开启仿真预览时自动运行会**拒绝启动**。
- **Detector tuning**：叠加 YOLO 检测框的实时头部相机画面。该页面与机器人共用同一个 `YoloGate` 实例，因此在此修改置信度会立即作用于正在运行的机器人。
- **VR teleop**：Pico 遥操作与示教录制。
- **Evaluation**：基于 `infer_logs/eval/*.jsonl` 的实时成功率看板。

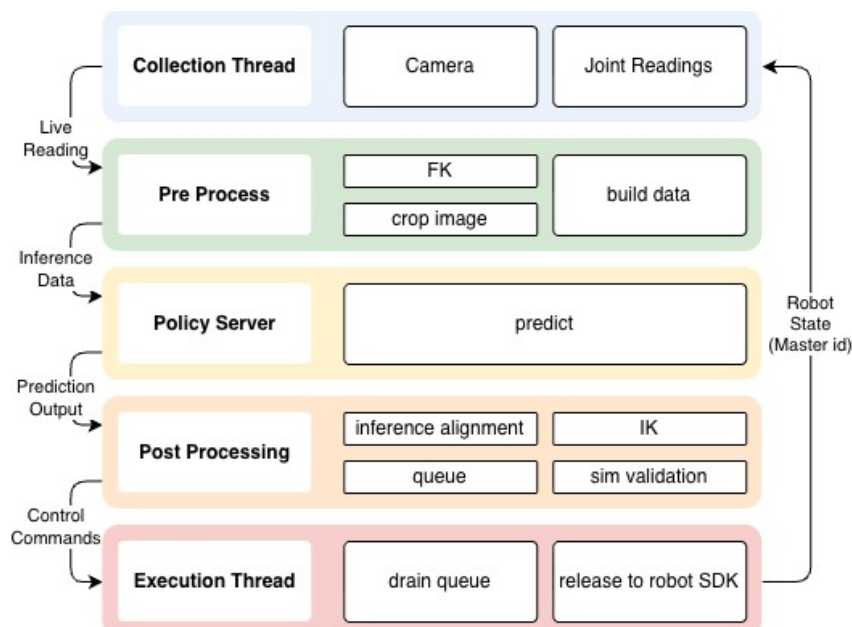
除自动运行外还有一条手动单步流程 (① Infer once → ② Validate → ③ Release *n*)：只要不按 Release，代码与机器人之间就是完全隔离的。

SDK 解耦

SDK 与 ROS 仅安装在机器人主机上。IK、仿真、相机集线器、录制观测回放、时序、dataset 构建以及整个测试套件，均可在缺少二者的环境下正常导入。实现手段有三：惰性导入 (`real_world/__init__.py`，模块级 `__getattr__`)、受保护的 SDK 导入 (`real_world/humanoid_env.py:62`)，以及依赖注入 (仿真运行器注入 `NoRobot` 替身)。仿真、评测与 CI 能够在笔记本上运行正是源于此；迭代速度因此提升近一倍，大部分调试完全无需占用机器人。

3. 宏观模型

本代码的整体流程如下：



观测与动作互为镜像。 `real_world/build_data.py` 构造的正是 policy 训练时的数据布局，`real_world/postprocess.py` 则是其逆过程。训练侧 (`MDM_data_collection/build_dataset.py:48`) 直接从 `build_data` 导入同一批函数，因此训练与部署的一致性是由结构保证的，而非依赖人工同步两份副本。目前二者仅存的差异是图像编码方式（训练用 mp4，推理用 JPEG）。改动任一侧就必须同步改动另一侧，并且必须重新训练。绝大多数「model 变差了」的问题，追查到最后都是这两侧发生了偏离。

时序只归一个模块管。 `real_world/timing.py` 是 `RECORD_HZ`、`CONTROL_HZ`、`SPEED_SCALE`、`MAX_JOINT_VEL`、`MAX_JOINT_STEP`、`RAMP_JOINT_STEP` 和 `WATCHDOG_MAX_JOINT_JUMP` 的唯一真源。它的 docstring 记录了这么做的缘由：这些常量以前分散在三处（`humanoid_env`、`inference_controller`、`robot_data_collect`），久而久之就对不上了；同时执行时序与录制速度相互脱钩——行间隔在关节空间里细分，却按时间排空，导致每行的实际耗时取决于关节走了多远，大动作太慢、小动作太快。任何假定子步在时间上均匀分布的合并逻辑，都会被这一点破坏。不要在任何地方引入第二个频率常量。

修复方案已固化在常量定义中：每个 policy row 恰好展开为 `SUBSTEPS_PER_ROW = (CONTROL_HZ / RECORD_HZ) / SPEED_SCALE` 个时间上均匀的子步。行与行之间的实际耗时恒为 `ROW_DT`，与运动幅度无关；平滑程度仅由 `CONTROL_HZ` 决定。`MAX_JOINT_STEP` 自此仅作为安全上限，不再是调节平滑度的旋钮。

对齐依据的是绝对 master row ID，而非时钟。每个 action row 都携带一个绝对 ID，它构成机器人自身的执行时钟。锚定在 `S` 的 chunk，其第 j 行的 ID 即为 $S + j$ 。释放循环在弹出的同时推进该时钟（`postprocess.py:611`），`queue_status()` 对外发布（`current_row_id`，`queued_through`），每次新推理均以这两个值为依据做合并。该方案可以容忍推理延迟波动，而基于时钟的同步方案不行（详见 §7.2）。

安全防护分层实现，且不依赖 policy。相关不变式在代码中记为 C1–C7 / H1–H4，由 `tests/test_safety_invariants.py` 固化保证：

ID	防护	位置
C1	仿真未运行时，任何指令都无法下发到机器人	<code>postprocess.py:642</code>
C2	每条释放的轨迹均已在仿真中完整执行过，并通过自碰撞检查	<code>postprocess.validate_chunk</code>
C3	急停为锁存式，会主动保持位姿，未复位则拒绝释放	<code>humanoid_env.py:734</code>
C4	关节读数异常时回退到上一次有效值，运动过程中绝不返回 <code>None</code>	<code>humanoid_env.py:859</code>
C5	每个下发子步 $\leq$ <code>MAX_JOINT_STEP</code> （双臂）；超限则转为斜坡下发	<code>humanoid_env.py:1173</code>
C6	单条指令跳变超过 <code>WATCHDOG_MAX_JOINT_JUMP</code> (0.5 rad) 即锁存急停	<code>humanoid_env.py:1138</code>
C7	指令末端执行器超出由数据估计的安全包络即锁存急停	<code>humanoid_env.py:1149</code>
H1	一次校验只允许释放一次，不会因重复释放而回弹	<code>humanoid_env.py:226</code>
H2	传感器数据陈旧或冻结时直接中止本次推理，不强行预测	<code>inference_controller.py:275</code>
H3	姿态在行与行之间做 SLERP 平滑（ <code>QUAT_ALPHA = 0.5</code> ）	<code>postprocess.py:68</code>
H4	各臂工作空间 AABB 在进入 IK 之前即拦截越界目标	<code>postprocess.py:70</code>
–	每个释放周期轮询固件错误与碰撞，触发即急停	<code>humanoid_env.py:446</code>

C6 和 C7 的分工值得记住：C6 拦的是大幅旋转，C7 拦的是缓慢漂移。只有 C5 是不够的——它约束的是速度，因此一次错误的 IK 分支翻转照样会被执行，只是执行得慢，表现为机械臂缓慢转过一个巨大的角度。C6 约束的是单条指令的位移，可以直接拒绝这类指令。

没有请求就没有数据流。 `CameraHub`（`real_world/camera.py`）为每台相机持有一个 SDK 相机对象，仅在消费者调用 `request()` 时才发起订阅，空闲 `CAMERA_IDLE_TIMEOUT = 5.0 s` 后自动释放。

4. 模块说明

建议按下表顺序阅读。行数仅作为模块分量的粗略参考。

模块	行数	职责	何时阅读
<code>real_world/timing.py</code>	102	全部频率与限值常量，以及其取值依据	首要必读。

模块	行数	职责	何时阅读
<code>real_world/build_data.py</code>	152	观测到 <code>/predict</code> 请求的构造；共享的像素与末端执行器行变换	改动任何 model 可见的内容时
<code>real_world/postprocess.py</code>	786	完整输出流程、机器人队列、合并缓冲区	排查运动质量、平滑度、拼接问题
<code>real_world/ik.py</code>	354	Pinocchio DLS IK、单臂模型、rot6d 与 四元数互转、标定加载	排查 IK、坐标系、不可达目标
<code>real_world/humanoid_env.py</code>	1225	SDK 归属、3 个线程、下发防护、夹爪锁存、录制	排查硬件行为、急停、线程问题
<code>real_world/inference_controller.py</code>	874	自动循环、服务器往返、宏调用、追踪	排查循环顺序、宏的触发时机
<code>real_world/observer.py</code>	209	观测缓冲、新鲜度判定（H2）、 <code>get_obs</code>	排查陈旧观测导致的中止
<code>real_world/camera.py</code>	232	动态相机订阅集线器	排查带宽问题、画面收不到
<code>real_world/sim_backend.py</code>	683	PyBullet 世界与 <code>validate()</code>	排查校验失败
<code>real_world/sim_preview.py</code>	99	纯仿真预览循环，完全不接触机器人	–
<code>real_world/recording.py</code>	206	回合录制器（mp4 + npz + 元信息）	数据采集
<code>real_world/recorded_obs.py</code>	127	将录制作为观测源回放，不依赖 SDK	离线评测

4.1 数据格式

发送至服务器的观测（`build_data.py:143`，与 `task/dual_arm_ee_image.yaml` 对应）：

字段	形状	内容
<code>agentview_image</code>	(To,) b64 JPEG	头部相机，从顶部裁切为 16:9，保留画面下方的作业区
<code>robotl_eye_in_hand_image</code>	(To,) b64 JPEG	左腕，中心裁剪
<code>robotr_eye_in_hand_image</code>	(To,) b64 JPEG	右腕，中心裁剪
<code>robotl_eef_pos</code>	(To, 9)	左末端执行器 <code>[pos(3) + rot6d(6)]</code>
<code>robotr_eef_pos</code>	(To, 9)	右末端执行器 <code>[pos(3) + rot6d(6)]</code>
<code>robot0_grip</code>	(To, 2)	<code>[left, right]</code> 夹爪，原始值，与录制时存储的一致

服务器返回的动作：共 20 列，格式为 `L[pos3, rot6d6, grip1] ++ R[pos3, rot6d6, grip1]`，夹爪位于第 9 列与第 19 列。10 列的仅左臂数据行仍可通过校验，用于兼容旧的单臂 policy。

所有图像先裁成 16:9，再缩放到 `IMG_W × IMG_H = 256 × 144`：无论训练还是推理、无论哪台相机，走的都是同一个 `preprocess_frame`（`build_data.py:71`）。裁剪在 JPEG 编码之前完成，因此 `imencode` 需处理的像素量仅为原生 1280×800 头部画面的约 1/25。若将 `AGENT_CROP_ZOOM` 提高到 1.0 以上，服务器端必须关闭自身的中心裁剪，否则头部画面会被裁切两次。

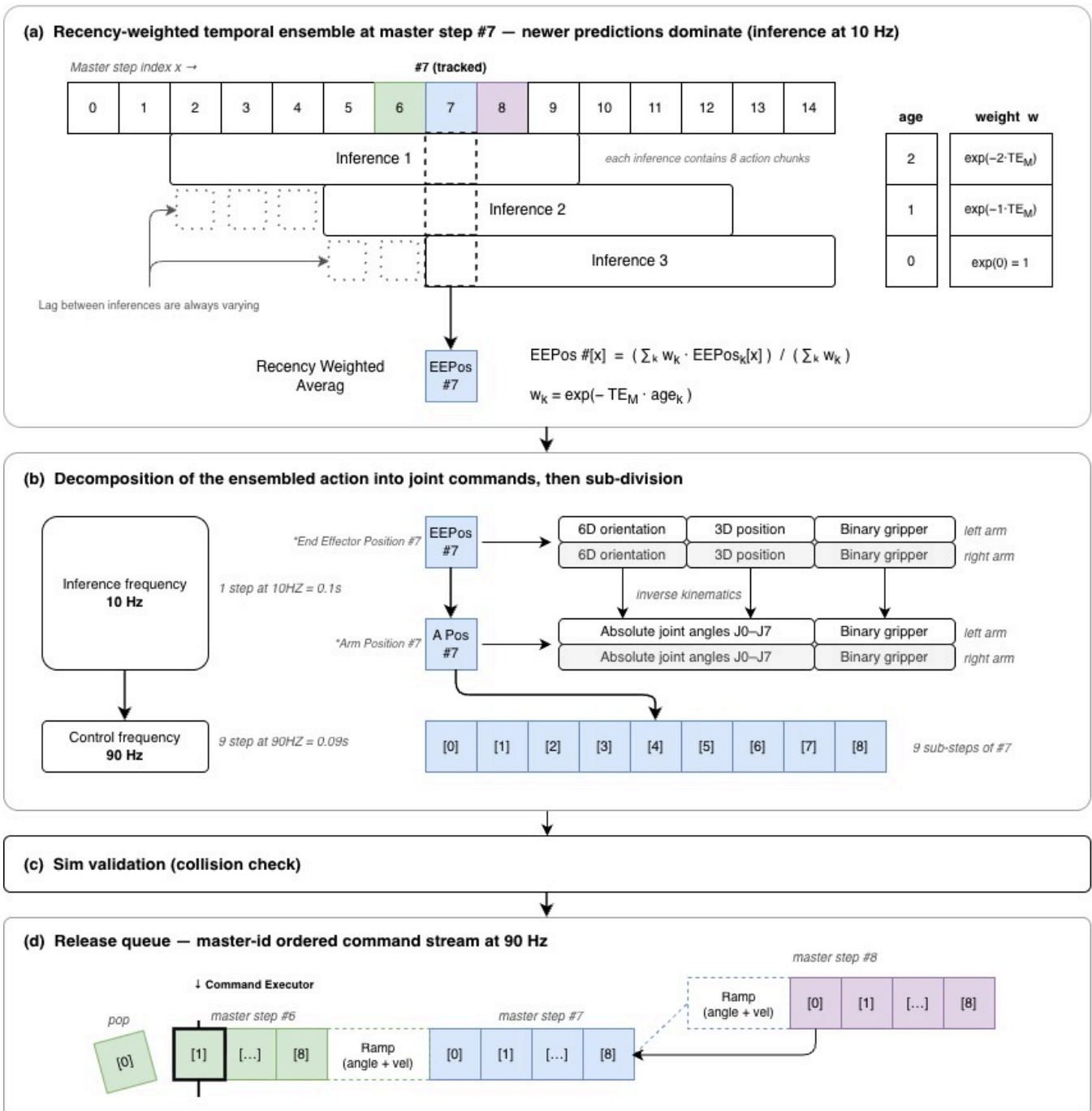
4.2 输出流程（`postprocess.py`）

1. **夹爪二值化**（`:230`）：以 `GRIPPER_CLOSE_THRESH = 10.0` 为阈值，将原始值 `[0, ~85]` 映射为 `{0, 1}`。chunk 一到达即就地完成，避免夹爪毛刺污染下一次推理的状态上下文。
2. **temporal ensemble 合并**（`:326`）：包含两个平滑维度，均以 master ID 为键。*(a)* 跨 chunk：将所有缓冲 chunk 中处于同一绝对 ID 的行按新近度加权平均（`w = exp(-TE_M · age)`），即 ACT 的 temporal ensemble；*(b)* 沿 ID 轴做半宽为 `TE_RADIUS` 的对称高斯平滑。ID $\leq$ `queued_through` 的行为冻结行（已提交给机器人，仅作为只读的左侧上下文，以保证接缝连续）；ID 更大的行为可变行，每次合并均重新构建。位置与 rot6d 采用线性平滑；夹爪绝不沿 ID 轴做低通（这会模糊开合时机），而是沿用跨 chunk 的结果并以 0.5 重新取阈。参与平均的必须始终是原始 chunk，不可将集成后的输出重新写回缓冲区，否则平滑会逐层叠加。

3. **双臂 IK (:463)**: 依次执行逐行工作空间闸门 (H4)、姿态 SLERP (H3) 与各臂 IK, 行与行之间热启动。**_ik_robust (:443)** 的逻辑值得理解: 先以实时种子或链式种子求解, 若解不可达或贴近关节限位 (任一关节距限位不足 0.10 rad, 这正是将冗余自由度顶到限位的扭曲解的典型特征), 则改用训练时的标称姿态重试 (`config/nominal_arm_config.json`, 取自 67 106 帧录制数据的中位数)。正是这一步使保持静止的右臂能求出合理解而不产生多余运动; 仿真评测始终未能暴露该问题, 因为它总是从热启动的录制关节值出发。
4. **仿真校验 (:512)**: 在 PyBullet 中逐子步执行一遍, 进行自碰撞检查, 再读回仿真实际达到的关节值。仿真会返回精确的逐子步索引, 因此即使某个受速度限制的行产生了多于 K 个子步, master ID 标记仍保持一行一 ID (由 `tests/test_tagging_exact.py` 固化保证)。
5. **队列拼接 (:631)**: `append_actions` 始终在时钟前方保留 `append_ahead_rows` 个 policy row, 且只追加尚未入队的 ID。行只追加、不清空; 释放循环仅从队首弹出。稳态下每次推理恰好追加一行, 无需桥接 (`seed_gap=True` 会让仿真展开队尾到第 0 行的间隙, 轨迹本身是连续的)。只有当校验耗时过长、时钟已越过 `start_id` 时, 才会触发按速度合并的桥接; 其巡航速度取队尾出口速度与新行入口速度的平均值, 以获得尽可能平滑的过渡。

Action-chunk execution pipeline

Fill colour = master step index: ■ step #6 ■ step #7 ■ step #8 dashed = ramp / interpolation between master steps



流式路径中一旦遇到不可达的行，整个 chunk 中止，机械臂原地保持（:674 处 skip_unreachable=False）。若中途丢行，机械臂会跳过路点直接跨越，表现为运动生硬。代码注释已将该项标记为可 bisect 的候选；后续若要重新优化运动平滑度，这是一个已知的可调杠杆。

auto_ingest_chunk（:727）是更早的队列替换版本，实际生效的是 append_actions。

4.3 夹爪处理

三套机制各司其职，很容易混淆：

- **观测侧：** 夹爪以非二值形式输入 model，让 policy 看到真实的连续状态；但输入的是指令值而非固件回读值（humanoid_env.py:244, _grip_obs_from_command = True，闭合状态映射到训练时的原始量程 119.8）。回读值存在数秒滞后，曾导致 policy 反复下发早已完成的抓取动作。
- **动作侧：** 在后处理中二值化（§4.2 第 1 步）。输入端给出丰富状态、输出端给出明确决策，指令抖动因此大幅减少。
- **下发侧：** 一个防抖的变化锁存（humanoid_env.py:951）。某个通道的二值指令一旦翻转，该状态即被提交并锁定 20 个 master row ID，保持期内的反复切换一律忽略；这样即便 policy 出现振荡，也不会触发重复抓取。该锁存会在急停、复位、自动运行启动以及每次回退时由 reset_grip_latch() 清除。一旦漏清，就会出现「锁存为闭合的抓取」悄悄覆盖一次本应张开的指令。

5. 数据与训练

5.1 表示方式

	表示	为什么
输入	3D 位置 + 四元数 + 夹爪（后期追加关节角）	四元数作为输入紧凑且高效。追加关节角是因为纯末端执行器表征的 model 无法感知机械臂构型。
输出	3D 位置 + 6D 旋转 + 夹爪	6D 旋转表示连续，对训练稳定性至关重要；四元数存在双重覆盖且不连续，作为回归目标表现很差。

采用末端执行器空间而非关节空间，是 diffusion policy 作者本人的建议。关节角输出也做过训练和测试，末端执行器输出的效果更好。（若需重新启用，关节空间的 dataset 构建脚本仍保留在 MDM_data_collection/build_dual_arm_replay_buffer.py。）

5.2 从录制到 dataset

录制以 30 Hz 采集（MDM_data_collection/robot_data_collect.py:15），dataset 每 3 帧抽 1 帧，构建为 10 Hz。这样每个 action row 覆盖的运动量增至三倍，model 每次推理能向前推进更长的距离；§7.3 中的停滞问题正是由此解决的。

dataset 会将自身的构建频率写入 meta/record_hz。当 --fps 与 timing.RECORD_HZ 不一致时，build_dataset.py:331 会直接拒绝构建，除非显式传入 --allow-hz-mismatch。这条不变式过去仅存在于注释中，请保持其强制性。观测行保持原始值，只对动作目标做高斯平滑（30 Hz 下 $\sigma = 1.7$ 个输出帧，并随 --fps 自动缩放）。

5.3 要多少数据、训多少轮

dataset 规模	表现
20 组	动作大致可复现，但一接近物体鲁棒性即崩溃；初始对位一旦有偏差就无法挽回。
50 组	单臂行为达到可用水平。
100 组	示教中的细节均可复现，连「先松开、用夹爪下缘把包裹推正再抽手」这类个人操作习惯都能学到。
200 组	当前 dataset，已有意纳入更困难的抓取工况。

约 50 组示教配合 50–100 epoch，即可基本复现动作。超过约 100 epoch 后收益明显递减：最长的一次训练跑到 200 epoch，未观察到进一步提升。没有明确理由，不要继续投入 GPU 时间。

5.4 采高质量数据的要点

本节是全文价值最高的部分。model 的上限取决于示教操作的一致程度。这些经验在代码中没有任何体现，仅记录于此。

- **动作要夸张。** 细微动作无法在训练中保留下来。遥操作时感觉幅度大得没有必要，映射到机器人上往往刚好合适。
- **意图要明确表达。** 端到端视觉 model 不存在「隐含意图」，它只会学到你反复做出的那个模式。
- **接近余量要一致。** 具体而言，若包裹在头部相机中占据 x_1 至 x_2 、 y_1 至 y_2 这一区域，就始终让夹爪从 $x_1 - n$ 与 $x_2 + n$ 处接近， n 取固定值，约 5 cm 效果较好，且每次都保持一致。当包裹紧贴台面、位置偏低、夹爪几乎没有余隙时，这一条尤为关键。**该环节不一致，是抓取失败的最大单一来源。**

5.5 图像分辨率

当前设置：全部裁成 16:9 后缩放到 256×144，头部相机从顶部裁切（保留画面下方的作业区），腕部相机居中裁切。每台相机配备独立的 ResNet 编码器（服务器端），设计思路是由头部相机负责粗定位，腕部相机负责对准与夹爪决策。

6. 设计决策和它们的由来

6.1 policy 加脚本化宏的混合方案

policy 负责抓取、抬升与翻转，其余环节全部交由内联在自动推理循环中的脚本化宏处理，执行顺序如下 (inference_controller.py:803-858)：

```
E-stop check → package gate → no-flip place → flip place → recovery (inside _run_inference) → predict
```

模块	干什么	什么时候触发
package_gate.py	无包裹时暂停推理，将机械臂停靠归位	YOLO 检测不到 package 类别，且仅在空闲状态下生效
no_flip_place.py	原样放置：将录制的关节路径映射到当前位姿，执行移出、张开、原路退回	YOLO 连续 20 次检出 barcode，且已持续抓握 1.2 秒
flip_place.py	同上，用于 policy 完成约 180° 翻转之后	右腕滚转自抓取起累计转过 ≥ 2.5 rad，并保持 0.8 秒
grasp_recovery.py	抓空处理：清空队列、张开、回退，交由 policy 重新规划	右夹爪指令为闭合时，腕部 YOLO 报告 closed-empty
retreat.py	不依赖 torch、受速度约束的回退原语，供上述模块共用	由以上模块调用

两个放置宏共用一套设计，值得理解清楚 (flip_place.py:119)：

- 在关节空间实现，而非末端执行器空间。固定的终点关节构型经正运动学即可保证落在同一释放点，完全不需要 IK：既不会出现冗余分支翻转，腕部也不会转错方向。
- 起点自适应，终点固定。以递减偏移对录制轨迹的形状做形变并重新锚定，使 $out[0] = \text{当前位姿}$ 、 $out[-1] = \text{录制的释放构型}$ ： $out[i] = rec[i] + (q_now - rec[0]) \cdot (1 - i/(M-1))$ 。
- 左臂全程保持不动，锁定在当前位姿 ($fwd[:, :7] = q_now[:7]$)。
- 执行前后均需清空机器人队列、暂存区、流式游标、合并缓冲区与夹爪锁存，否则自动运行恢复时会出现突跳。以上任何一项遗漏都曾引发过故障。
- 路径文件由 scripts/build_release_path.py 从真实录制生成 (recording205 对应不翻转，recording206 对应翻转)，按可达范围裁剪并做轻度平滑，端点固定。

这样划分并非为了省事。放置动作几何约束明确、重复性极高，脚本化实现反而更可靠，成本也远低于学习该动作所需的示教量；policy 的容量应当用在真正困难、接触密集环节上。评审时应能为这一取舍给出充分理由，同时也要承认其局限：不重写脚本就无法泛化到新的放置位置。

6.2 替换原生 SDK 的 IK

SDK 自带一个黑箱 IK 求解器 (`set_end_effector_pose_control`)。我们将其替换为透明的 URDF + Pinocchio 阻尼最小二乘 (DLS) 求解器 (`real_world/ik.py`)：每条臂使用一个降阶的 7 自由度模型、其余关节锁定，再叠加一个恒定的 `base_offset` SE3 变换，将 URDF 的 `base_link` 映射到固件参考坐标系。

标定由 `scripts/fk_consistency_check.py` 基于录制回合离线拟合：若单个常量 X 能在所有帧上满足 $SDK_FK(q) = X \cdot our_FK(q)$ ，则说明固件坐标系实际是相对各臂定义的，离线标定成立。随仓库提供的拟合结果如下：

	位置残差	姿态残差
<code>config/fk_calibration.json</code> (左臂)	0.10 mm	0.008°
<code>config/fk_calibration_right.json</code> (右臂)	0.39 mm	0.116°

此外还做过独立验证：将录制数据回放通过该求解器，在 400 多个目标点上最大偏差低于 8 mm 与 0.03 rad。

最关键的收益在于可控性。相较于行为不确定的黑箱求解器，同一条 Pinocchio 轨迹可以先在 PyBullet 中完成校验，再原封不动地下载到实机。

有两处求解器细节耗费了较多时间。其一，DLS 步长被限制为每次迭代 0.2 rad (L2 范数) (`ik.py:252`)：雅可比接近奇异时，原始步长可达数弧度并导致发散，解会落到偏离目标一米开外、姿态相差约 360° 的位置。其二，求解器返回的是迭代过程中的最优解而非最后一次迭代结果，因为迭代在最优解附近会来回振荡，面对不可达目标时还会停在工作空间边界上。后期曾尝试放宽容差，反而使抖动重现，因此又恢复了严格的 IK 容差 (`reach_pos_tol = 0.02 m`)。

6.3 视觉：为什么采用 YOLO

快递单检测先后试过四种方案：

- 1. 条码库 (zxing-cpp)：极不稳定，检测噪声很大。判据逻辑本身没有问题（快递单上必然有条码），问题出在检测器。`BarcodeGate` 目前仍作为可替换接口保留在 `no_flip_place.py:359`。
- 2. ArUco 标记：仅用于端到端验证流程，从未作为正式方案。
- 3. 印刷文本块检测：传统的基于规则的做法。虽然在调参上投入不少（最大检测面积过滤、噪声下限、裁掉噪声较大的顶部条带，甚至专门开发了一整个实时 Detector Tuning 页面用于调节），效果仍不及条码方案。
- 4. YOLO：最终采用的方案，使用小模型即可（如 `yolov8n`）。在几乎不增加计算负担的前提下，检测稳定性相比前几种方案提升约一个数量级。

检测逻辑：

干什么	类别	谁在用	权重
判断包裹是否存在 (闸门)	<code>package</code>	<code>package_gate.py</code>	<code>real_world/assets/head_yolo.pt</code>
快递单检测 (是否需要 翻转)	<code>barcode</code>	<code>no_flip_place.YoloGate</code>	与上共用同一 model
夹爪状态 (三分类)	<code>open / closed-gripped / closed-empty</code>	<code>grasp_recovery.py</code>	<code>real_world/assets/right_yolo.pt</code>

7. 排障历程

7.1 末端执行器数据卡住，不更新

机器人状态陈旧有多个成因：

- 深度相机占满带宽 (约 600 MB/s)，挤占了其他 DDS 数据流。为此实现了按需订阅的 CameraHub：仅在需要时订阅相机，闲置一段时间后自动释放。
- SDK 提供的末端执行器位姿不可靠：关节状态正常更新，上报的末端执行器位姿却是冻结的。改用实时关节角经正运动学计算末端执行器位姿后解决。

- 关节状态冻结：关节状态话题只有在同一进程内存在 Slam() 实例时才会正常推送。因此当前环境中构造 Slam() 实例，纯粹是为了维持关节数据的实时推送。

7.2 运动振荡

振荡是通过一系列修复逐步压下去的：

- 修正 SDK 控制方法并降低单步运动量，剧烈抖动随之消失。
- 通过二值状态锁存稳定夹爪指令。
- 确认轨迹合并（无论按时间还是按最近状态）必然引发振荡。将预测轨迹绘制出来后才定位到根因：新轨迹落在已执行运动的后方。
- 以带高斯平滑的缓冲流程取代显式轨迹合并，随后进一步引入绝对 master ID 时序体系。
- 统一轨迹插值，修正速度缩放不一致的问题，并使桥接速度在轨迹交界处两侧对齐。
- 将高开销日志与冗余的 DDS 轮询移出 120 Hz 控制循环，实时性能得以恢复。

最终稳定的配置为：120 Hz 控制循环 + 高斯平滑轨迹缓冲，相关参数保存在 `tuning_config.json` 中。

7.3 推理频率

30 Hz 的 policy 在两次推理之间产生的运动量过少，导致机器人停滞。改用 10 Hz、以末端执行器为目标重新训练后问题解决；10 Hz 现已成为标准的录制与推理频率。

7.4 失败恢复

恢复机制从最初单一的 home 位姿，逐步演进为一整套流程：

- 每次运行前自动归位。
- 基于 YOLO 的抓取成功判定。
- 抓取失败后自动恢复。
- 恢复后重置合并缓冲区，使推理能够正常衔接。
- 降低后退速度，使动作更平顺、更安全。
- 五个自适应回路点，根据机器人在抓取流程中所处的阶段选取，相比每次都退回单一 home 位姿更节省时间。

8. 运行与调参参考

8.1 每个旋钮到底管什么

旋钮	默认值或实时值	作用	位置
CONTROL_HZ	120	运动分辨率的主旋钮。取值越高，每行的子步越多、运动越精细。必须为 RECORD_HZ 的整数倍。可在 SDK 能稳定承受的路点下发速率范围内上调（需在实机上验证）。	timing.py:46
RECORD_HZ	10	不可随意调整，取决于 policy 的训练方式。	timing.py:44
speed_scale	1.40 (JSON)	相对示教速度的比例。取值越低，每行子步越多：更平滑但更慢，chunk 持续时间更长，队列不易饥饿。由于对齐以 ID 为键，该值取多少都不会破坏对齐。	GUI 「Speed (x demo)」
append_ahead_rows	5 (JSON)	在时钟前方预排队的行数。约束条件为 $n \cdot \text{SUBSTEPS_PER_ROW} \cdot \text{STEP_TIME}$ 必须大于一次推理往返耗时，否则队列会饥饿、机械臂会停滞。取值越大越能容忍延迟，但响应也越迟钝。	GUI 「Look-ahead (rows)」

旋钮	默认值或实时值	作用	位置
te_radius	6	沿 ID 轴的高斯半宽，同时也是保留作上下文的冻结行数。	GUI 「Smoothness」
te_sigma	1.4	高斯 σ ，需保持在 te_radius 以内，否则核会被截断。	GUI 「Blend strength」
te_m	0.123	新近度衰减系数。取值越大越信任最新的 chunk（响应更灵敏，但也更毛糙）。	GUI 「Recency」
te_buffer_len	8	参与平均的最近原始 chunk 数量，即最大重叠深度。	GUI 「Overlap depth」
INFERENCE_HZ	0	取值 ≤ 0 时背靠背连续推理，chunk 重叠最多、TE 平均最充分。设为正值可降低服务器负载，代价是平滑度下降。	inference_controller.py:51
MAX_JOINT_VEL	4.0 rad/s	仅作为安全上限。真实示教运动峰值约 5 rad/s (p99.9 $\approx$ 2.7)，偶发的约 17 rad/s 属于传感器毛刺。	timing.py:67
WATCHDOG_MAX_JOINT_JUMP	0.5 rad	C6 的判定阈值。取值越低越严格，设为 0 即禁用。	timing.py:79

8.2 配置与资源文件

文件	内容	怎么重新生成
real_world/config/fk_calibration{,_right}.json	各臂从 URDF 到固件的 base_offset SE3	scripts/fk_consistency_check.py --side left right
real_world/config/nominal_arm_config.json	训练姿态中位数，IK 的回退种子	录制数据中 arm_joints 的中位数
real_world/config/retreat_waypoints.json	双臂各 5 个接近路点	scripts/estimate_retreat_waypoints.py
real_world/assets/{flip,no_flip}_release_path.npy	(M,14) 脚本化释放路径	scripts/build_release_path.py <recording> <out>
real_world/assets/head_yolo.pt	barcode 加 package 检测器	标注后训练 YOLO
real_world/assets/right_yolo.pt	三分类夹爪状态检测器	标注后训练 YOLO
real_world/assets/A2D_0mnipicker/A2D.urdf + meshes/	内置 URDF，自包含	厂商提供
postprocess.EE_SAFE_REGION_*	C7 安全包络，由数据估计并外扩 0.12 m	scripts/estimate_ee_region.py → 将输出的元组回填至代码
tuning_config.json	操作者实时调参结果	GUI 在每次滑块变化时写入

8.3 诊断手段

所有追踪日志都落在 `infer_logs/`（可用 `HUMANOID_TRACE_DIR` 覆盖）。

信号	位置	拿来看什么
<code>[infer] #N ... carried ids A..B robot@id C (queued→D, lead L) L-gap X cm</code>	控制台，每次推理	判断自动运行是否让机械臂跳离当前位姿。间隙偏大说明锚定点已偏离当前状态。 Lead 为已排队的超前行数。
<code>[release-timing] N substeps/s total = recorder + firmware + dispatch</code>	控制台，约 1 Hz	若 <code>total</code> 远大于 <code>STEP_TIME</code> (8.3 ms)，则限制机械臂速度的是释放循环而非推理。
<code>[pipeline] append: ... (+N catch-up)</code>	控制台	出现非零的追赶桥接，说明校验已落后于时钟。
<code>SMOOTHNESS WARN: buffer Δpos max=...</code>	日志	接缝附近单行末端执行器步长超过 3 cm，说明合并存在问题。仅用于诊断，不做任何钳制。
<code>buffer.jsonl</code>	始终开启	实际下发给机器人的平滑轨迹，按 <code>master ID</code> 排列。 建议绘图查看 。
<code>requests.jsonl</code> 、 <code>chunks.jsonl</code>	<code>HUMANOID_INFER_TRACE=1</code>	发出的原始本体感知数据，以及返回的原始 <code>chunk</code> 。
<code>released_substeps.jsonl</code> 、 <code>live_joints.jsonl</code>	<code>HUMANOID_SUBSTEP_TRACE=1</code>	每个 <code>master ID</code> 的指令关节值与实测关节值，可据此计算跟踪误差。 会占用实时预算 。
<code>scripts/analyze_smoothness.py [DIR]</code>	离线	将重叠日志折叠为每个 <code>master ID</code> 一个值；以单 <code>chunk</code> 为基线（ <code>policy</code> 自身的平滑度即可达上限）衡量实际执行的平滑度，并标出接缝处的速度尖峰。

8.4 不用机器人的离线评测

脚本	效果
<code>scripts/sim_replay_eval.py serve / send</code>	使用我们的 IK，按录制的末端执行器轨迹在 PyBullet 中驱动机械臂。无需 <code>policy</code> 服务器。
<code>scripts/sim_infer_eval.py --source replay policy</code>	用一段录制回合完整跑通部署路径（观测 → 服务器 → IK → 仿真）。
<code>scripts/sim_model_eval.py</code>	在录制的感知数据上，让训练好的 <code>policy</code> 在仿真中驱动双臂，输出 IK 可达性、各臂姿态误差、子步最大速度与上限的对比，以及预测末端执行器与录制值的偏差。 判断 <code>model</code> 是否正常，主要依据该脚本 。
<code>scripts/fk_consistency_check.py</code>	校验我们的 URDF 正运动学与固件正运动学是否一致，并顺带产出标定结果。
<code>scripts/eval_trials.py --task flip_place</code>	实机试验打分（SPACE = 开始，1..N = 结果）。结果写入 <code>infer_logs/eval/*.jsonl</code> ，并打印带 Wilson 95 % 置信区间的成功率及失败模式直方图。全程不与机器人交互。
<code>pytest tests/</code>	C1—C7 / H1 安全不变式、追加与拼接的连续性、 <code>master ID</code> 精确标记、回退路点选择。

最后更新：2026 年 7 月 31 日